

Setor de Educação Profissional e Tecnológica

Tecnologia em Análise e Desenvolvimento de Sistemas

DS122 - Desenvolvimento de Aplicações Web 1

05 - CSS3: layout e responsividade

Prof. Alex Kutzke

Setembro de 2026

Objetivos da aula

Ao final desta aula você deve ser capaz de:

1. Dizer o que muda nos filhos quando o pai recebe `display: flex` ou `display: grid`;
2. Montar uma barra de navegação horizontal com Flexbox;
3. Montar uma grade de cartões com Grid e calcular quantas colunas ela terá em uma largura dada;
4. Escolher entre Flexbox e Grid justificando pela dimensão do problema;
5. Escrever *media queries* em *mobile-first*;
6. Verificar a página a 320px e corrigir a rolagem horizontal;
7. Ler e alterar HTML que usa a grade e as utilitárias do Bootstrap 5;
8. Usar o inspetor de Flexbox e de Grid do navegador.

Roteiro

Fluxo

Flexbox

Grid

Responsividade

Bootstrap

Prática

Onde a página parou

O `catalogo.html` da tarefa passada tem borda, padding e cor de fundo em cada cartão.

E os cartões continuam empilhados, um debaixo do outro, ocupando a largura inteira.

Nada de errado aconteceu: isso é o **fluxo normal**.

Três declarações no inspetor

Com F12 aberto, na regra do elemento que contém os cartões:

```
display: flex;                /* mesma linha */
gap: 20px;                    /* afastados */

display: grid;                /* três colunas */
grid-template-columns: repeat(3, 1fr);
```

Depois, Ctrl+Shift+M liga o modo responsivo. Arraste até uma largura de celular e observe o texto transbordar.

O que apareceu na tela

- o elemento onde você escreveu o `display` passou a mandar na posição dos filhos;
- ao lado do seletor surgiu um distintivo `flex` ou `grid`, que liga um desenho por cima da página;
- a largura da janela virou uma variável do problema.

A aula nomeia essas três coisas, e volta ao inspetor no fim.

Por que os cartões empilham

Fluxo normal: caixas block uma abaixo da outra, na ordem do documento, ocupando toda a largura; caixas inline lado a lado, quebrando quando falta espaço.

`<article>`, `<section>` e `<div>` são block.

Até meados dos anos 2010, pôr dois blocos lado a lado exigia float, criada para texto contornar imagem, ou tabelas de diagramação. As duas soluções eram frágeis e exigiam remendos.

Contêiner e item

```
<div class="grade">           <!-- contêiner -->
  <article class="produto">    <!-- item -->
    <h3>Café em grão</h3>    <!-- não é item -->
    <p>Torra média.</p>
  </article>
  <article class="produto">    <!-- item -->
    </article>
</div>
```

O **contêiner** recebe display: flex ou display: grid. Os **itens** são os **filhos diretos** dele. Netos voltam ao fluxo normal dentro do próprio pai.

Duas famílias de propriedades

Propriedades que se escrevem **no contêiner** e propriedades que se escrevem **no item**.

O erro mais comum

`justify-content` dentro de `.produto` não faz nada, e o navegador não avisa. CSS inválido ou inútil é descartado em silêncio.

Quando uma propriedade de layout parecer ignorada, a primeira pergunta é se ela está no elemento certo.

Uma direção, e a sobra repartida

O Flexbox distribui itens ao longo de **uma** direção, e reparte entre eles o espaço que sobra ou que falta.

Propriedade	Atua no eixo
<code>justify-content</code>	principal
<code>align-items</code>	transversal

O eixo principal é definido por `flex-direction` (padrão `row`). Com `column`, as duas propriedades trocam de sentido. Guarde a relação pelo eixo, e não por horizontal e vertical.

Propriedades do contêiner

```
.barra {
  display: flex;
  flex-direction: row;           /* row | column | ... */
  justify-content: space-between; /* eixo principal */
  align-items: center;           /* eixo transversal */
  flex-wrap: wrap;               /* permite quebrar linha */
  gap: 1rem;                     /* espaço entre itens */
}
```

Os valores que aparecem na prática

justify-content

<code>flex-start</code>	itens no começo do eixo (padrão)
<code>center</code>	itens no meio
<code>space-between</code>	pontas nas bordas, sobra dividida entre eles
<code>space-evenly</code>	espaços iguais, inclusive nas pontas

align-items

<code>stretch</code>	esticam até a altura do mais alto (padrão)
<code>center</code>	centralizados na altura
<code>flex-start</code>	alinhados pelo topo
<code>baseline</code>	alinhados pela linha de base do texto

Duas armadilhas

`flex-wrap: nowrap` é o padrão

Sem `wrap`, os itens nunca quebram para a linha de baixo: eles encolhem até caber, e continuam encolhendo depois que o conteúdo já não cabe mais. Em cartões de produto, quase sempre você quer `wrap`.

`gap` não é `margin`

`gap` vale só *entre* os itens, e não sobra espaço nas pontas, que é o que a margem faria. Funciona em Flexbox e em Grid.

Propriedades do item

```
.item {  
  flex-grow: 1;      /* fatia da sobra (padrão 0) */  
  flex-shrink: 1;    /* aceita encolher (padrão 1) */  
  flex-basis: auto;   /* tamanho de partida */  
  align-self: center; /* alinhamento só deste item */  
}
```

A forma abreviada resolve quase tudo:

```
.busca { flex: 1; } /* = 1 1 0%: ocupe toda a sobra */  
.logo  { flex: 0; } /* tamanho do conteúdo */
```

Uma barra com campo elástico

```
.barra { display: flex; align-items: center; gap: 1rem; }  
  
.barra .logo    { flex: 0 0 auto; } /* não encolhe */  
.barra .busca   { flex: 1 1 auto; } /* absorve a sobra */  
.barra .entrar  { flex: 0 0 auto; }
```

Logotipo à esquerda, busca no meio ocupando o que sobrar, botão à direita.

O padrão da barra de navegação

```
header {  
  display: flex;  
  justify-content: space-between;  
  align-items: center;  
  padding: 1rem;  
}  
  
.menu ul {  
  display: flex;  
  gap: 1.5rem;  
  list-style: none; /* tira o marcador */  
  margin: 0;  
  padding: 0;      /* tira o recuo padrão da lista */  
}
```

Só as duas primeiras declarações do `ul` mudam o layout. As outras três a lista precisaria de qualquer forma.

order e a ordem de leitura

```
.destaque { order: -1; } /* aparece antes dos demais */
```

A ordem **visual** muda. A ordem do **documento** não.

Quem navega com Tab e quem usa leitor de tela continua percorrendo os elementos na ordem do HTML. Se as duas ordens discordarem, a pessoa vê o foco pular de um canto a outro da tela.

WCAG 1.3.2 (sequência com significado) e 2.4.3 (ordem de foco). Quando a ordem certa for outra, mude o HTML.

Linhas e colunas ao mesmo tempo

```
.grade {  
  display: grid;  
  grid-template-columns: 1fr 1fr 1fr; /* três colunas iguais */  
  gap: 20px;  
}
```

fr é uma fração do espaço livre do contêiner, descontados o gap e as colunas de tamanho fixo. Só existe dentro do Grid.

```
/* mesma coisa que 1fr 1fr 1fr */  
grid-template-columns: repeat(3, 1fr);
```

```
/* lateral fixa de 250px, e o resto para o conteúdo */  
grid-template-columns: 250px 1fr;
```

Um item que ocupa mais de uma célula

```
.produto.destaque {  
  grid-column: span 2;  /* ocupa duas colunas */  
  grid-row: span 2;     /* e duas linhas */  
}
```

É o recurso que o Flexbox não tem, e o motivo de o Grid existir.

`grid-template-rows` existe, mas raramente é necessária: as linhas que faltarem são criadas automaticamente, com a altura do conteúdo.

Áreas nomeadas: o esqueleto da página

```
.pagina {  
  display: grid;  
  grid-template-columns: 1fr 250px;  
  grid-template-areas:  
    "cabecalho cabecalho"  
    "conteudo lateral"  
    "rodape rodape";  
  gap: 1rem;  
}
```

```
header { grid-area: cabecalho; }  
main   { grid-area: conteudo; }  
aside  { grid-area: lateral; }  
footer { grid-area: rodape; }
```

O mesmo layout na tela estreita

Cada linha entre aspas é uma linha da grade. Nome repetido em células vizinhas estende a área. Um ponto marca célula vazia.

```
grid-template-columns: 1fr;  
grid-template-areas:  
  "cabecalho"  
  "conteudo"  
  "lateral"  
  "rodape";
```

Reescrever o layout é redesenhar o quadro.

A grade que se adapta sem media query

```
.grade {  
  display: grid;  
  grid-template-columns: repeat(auto-fit, minmax(240px, 1fr));  
  gap: 20px;  
}
```

Em português: crie quantas colunas couberem, com no mínimo 240px cada uma, e reparta a sobra igualmente entre elas.

Esta declaração resolve a grade de cartões do trabalho prático inteira.

A conta, e a medição

Com gap de 20px, cabem n colunas enquanto

$$n \times 240 + (n - 1) \times 20 \leq \text{largura do contêiner}$$

Em 1000px: 4 colunas dariam $960 + 60 = 1020$, não cabe. 3 colunas dão $720 + 40 = 760$, cabe. Cada uma recebe $(1000 - 40) / 3 = 320\text{px}$.

Contêiner	Colunas	Cada uma
1000px	3	320px
800px	3	253.33px
600px	2	290px
500px	2	240px
400px	1	400px

auto-fit e auto-fill

auto-fill cria as colunas vazias em vez de descartá-las.

Com poucos itens e muito espaço:

- auto-fill deixa os cartões no tamanho mínimo e sobra vazio à direita;
- auto-fit estica os cartões que existem.

Para catálogo, auto-fit costuma ser o que se quer.

Flexbox ou Grid

Flexbox quando	Grid quando
os itens ficam em uma linha ou em uma coluna	há linhas e colunas ao mesmo tempo
o tamanho de cada item vem do conteúdo dele	o desenho da grade vem primeiro
distribuir a sobra entre poucos itens	alinhar itens de linhas diferentes

Na página de catálogo: Grid no esqueleto e na grade de produtos; Flexbox no cabeçalho, no menu e dentro do cartão. Um contêiner Flex pode conter um Grid, e o contrário também.

Nos dois modelos, as margens verticais dos itens não colapsam.

A meta viewport

Sem esta linha no <head>, nada do que vem depois funciona no celular:

```
<meta name="viewport" content="width=device-width, initial-scale=1">
```

Sem ela, o navegador do celular finge uma janela de cerca de **980px**, desenha a página nela e reduz tudo por escala. Resultado: a página em miniatura, texto ilegível, e nenhuma *media query* de largura pequena entra em ação.

Não acrescente `user-scalable=no` nem `maximum-scale=1`: elas impedem o zoom com os dedos, e quem tem baixa visão depende disso.

Media queries

```
/* estilo base: telas pequenas */  
.grade { grid-template-columns: 1fr; }  
  
@media (min-width: 768px) {  
  .grade { grid-template-columns: repeat(2, 1fr); }  
}  
  
@media (min-width: 1024px) {  
  .grade { grid-template-columns: repeat(3, 1fr); }  
}
```

Dentro do bloco vão regras CSS comuns, com os mesmos seletores.

Media query não muda especificidade

As regras de dentro do bloco vencem as de fora por virem **depois** no arquivo, e não por serem mais específicas.

Consequência prática

As *media queries* vão ao **final** da folha. Escritas antes das regras que pretendem sobrescrever, elas perdem em silêncio.

Mobile-first

CSS base para a tela pequena, ampliado com `min-width`.

O caminho inverso, começar pelo desktop e cortar com `max-width`, também funciona, e produz folhas maiores: o layout simples costuma ser o da tela estreita, e ele fica sendo o caso que não precisa de regra nenhuma.

Há também a razão de origem: a maior parte do acesso à web vem de telefone, e frequentemente de aparelho modesto com conexão limitada.

O ponto de quebra vem do conteúdo

A tentação é copiar uma lista de larguras de aparelhos. Essa lista muda todo ano e nunca esteve completa.

O critério prático: arraste a janela do estreito para o largo e observe onde o layout fica ruim. Ali é o ponto de quebra.

Duas ou três quebras resolvem a maior parte das páginas desta disciplina.

Imagens e mídia fluidas

Uma imagem de 1200px de largura natural estoura qualquer coluna estreita e cria rolagem horizontal na página inteira.

```
img, video {  
  max-width: 100%;  
  height: auto;  
}
```

`max-width: 100%` impede a imagem de passar da largura do container. `height: auto` mantém a proporção enquanto ela encolhe.

O teste dos 320px

WCAG 2.2, critério 1.4.10 (*Reflow*, nível AA)

A página deve continuar utilizável, sem rolagem horizontal, em uma largura de **320 pixels CSS**.

Equivale a um celular pequeno, e ao que sobra quando alguém amplia a página em 400% num monitor comum.

Modo responsivo, largura 320, rolar a página de cima a baixo. Se aparecer barra horizontal, os suspeitos de sempre são imagem sem `max-width`, largura fixa em px, `white-space: nowrap` e tabela larga.

Layout, acessibilidade e os ODS

- grade que não se adapta obriga quem acessa pelo telefone a ampliar e arrastar a tela a cada linha de texto;
- ordem visual em desacordo com a do documento desorienta quem navega por teclado;
- página que só funciona em conexão rápida exclui quem não tem uma.

ODS 4 e ODS 10: o acesso à informação depende de a página funcionar no aparelho que a pessoa tem, e não no de quem a desenvolveu. **ODS 8:** interface que funciona no dispositivo de quem opera o sistema reduz retrabalho de quem trabalha com ela todo dia.

Um framework CSS

Folha de estilo pronta, escrita por terceiros, usada pelas classes que ela define. O Bootstrap é o mais difundido, e é o que a ementa pede que você conheça.

```
<link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.8/dist/css/bootstrap.min.css"
      rel="stylesheet"
      integrity="sha384-sRI14kxILFvY47J16cr9ZwB07vP4J8+LH7qKQnuqkuIAvNWLzeN8tE5YBujZqJLB"
      crossorigin="anonymous">
```

`integrity` traz o resumo criptográfico do arquivo esperado. Se o que chegar do CDN não corresponder, o navegador recusa o arquivo.

A grade de 12 colunas

.col-md-1	.col-md-1	.col-md-1	.col-md-1	.col-md-1	.col-md-1	.col-md-1	.col-md-1	.col-md-1	.col-md-1	.col-md-1	.col-md-1
.col-md-8								.col-md-4			
.col-md-4				.col-md-4				.col-md-4			
.col-md-6						.col-md-6					

A largura é dividida em 12 colunas. Você declara quantas cada bloco ocupa, e a soma de uma linha deve dar 12.

Grade e pontos de quebra

```
<div class="container">
  <div class="row">
    <div class="col-md-8">Conteúdo principal</div>
    <div class="col-md-4">Barra lateral</div>
  </div>
</div>
```

Sufixo	A partir de
(nenhum)	qualquer largura
sm	576px
md	768px
lg	992px
xl	1200px
xxl	1400px

Classes utilitárias

```
<div class="d-flex justify-content-between  
          align-items-center p-3 mb-4 bg-light">
```

d-flex é display: flex; p-3 é um padding da escala de espaçamento; mb-4 é margin-bottom; bg-light é uma cor de fundo.

Por baixo, .row é um contêiner Flexbox e .col-md-8 define largura em porcentagem dentro de uma *media query*. É o CSS desta aula, empacotado.

O que isso custa, e a posição da disciplina

O `bootstrap.min.css` 5.3.8 tem **232.111 bytes**, cerca de 227 KB, que chegam como cerca de 30 KB comprimidos. A página carrega a folha inteira, mesmo usando cinco classes dela. E o HTML acumula classes de apresentação, que é o que a separação entre estrutura e estilo pretendia evitar.

Nesta disciplina

O Bootstrap não é ferramenta obrigatória, e o trabalho prático é feito com CSS escrito por você. Quem aprende `col-md-4` sem ter escrito uma *media query* fica sem saber o que fazer quando a grade do framework não resolve o caso.

Você vai encontrá-lo em código de terceiros e em estágio: o que se espera daqui é saber **ler**.

Roteiro da prática

Trabalhando sobre as páginas da tarefa de CSS:

1. meta viewport nas três páginas, e o menu na horizontal;
2. a grade de produtos com auto-fit e minmax;
3. o cartão por dentro, com o preço alinhado na base;
4. *media queries* e o teste dos 320px.

O texto completo do roteiro está no material da aula, na parte prática.

O menu na horizontal

```
header {  
  display: flex;  
  justify-content: space-between;  
  align-items: center;  
  gap: 1rem;  
  padding: 1rem;  
}  
  
.menu ul {  
  display: flex; gap: 1.5rem;  
  list-style: none; margin: 0; padding: 0;  
}
```

Dois experimentos, um de cada vez: trocar space-between por center; acrescentar flex-direction: column ao ul.

A grade de produtos

```
.grade {  
  display: grid;  
  grid-template-columns: repeat(auto-fit, minmax(240px, 1fr));  
  gap: 20px;  
}  
  
img { max-width: 100%; height: auto; }
```

Ligue o desenho da grade no inspetor e conte as colunas. Estreite a janela até a grade passar para duas colunas, anote a largura e confira a conta.

Depois troque 240px por 320px e observe onde ela muda.

O cartão por dentro

Descrições de tamanhos diferentes deixam o preço em altura diferente em cada cartão.

```
.produto {  
  display: flex;  
  flex-direction: column;  
  height: 100%;  
}  
  
.produto .preco {  
  margin-top: auto;  /* absorve a sobra vertical */  
}
```

`margin: auto` dentro de um contêiner Flex absorve o espaço livre daquele lado. É a forma mais curta de empurrar um item para a ponta do eixo.

Media queries e o teste final

```
header { flex-direction: column; }

@media (min-width: 768px) {
  header { flex-direction: row; }
}
```

Em cada uma das três páginas:

1. Ctrl+Shift+M para ligar o modo responsivo;
2. largura 320, e rolar a página de cima a baixo;
3. achar no inspetor o elemento desenhado além da borda direita;
4. repetir em 1200px e confirmar que o layout largo continua correto.

Exercícios

Entrega

Valem nota, no item *Exercícios em sala*. O **prazo** está na atividade correspondente na **UFPR Virtual**, e a entrega é por push no GitLab.

- enunciado no README.md do repositório-modelo, com o endereço na UFPR Virtual;
- individual ou em dupla, com *fork* dentro do grupo;
- atividade avaliativa: sem IA generativa para produzir o código;
- o que sair daqui é a **Entrega 1** do trabalho prático.

Para treinar fora da entrega, em português: Flexbox Froggy e Grid Garden.

Resumo: Flexbox e Grid

- os dois modelos entram quando o **pai** declara o `display`, e valem para os filhos diretos;
- propriedade de contêiner escrita no item não faz nada e não gera erro;
- `justify-content` no eixo principal, `align-items` no transversal, e `flex-direction` troca o sentido dos dois;
- `flex-wrap`: `nowrap` é o padrão: sem wrap os itens encolhem em vez de quebrar a linha;
- `repeat(auto-fit, minmax(240px, 1fr))` muda o número de colunas sozinho, sem *media query*;
- Grid para o esqueleto e para grades, Flexbox para uma linha ou uma coluna de itens.

Resumo: responsividade

- sem a meta viewport, o celular desenha numa janela fingida de 980px e nenhuma *media query* estreita entra em ação;
- em *mobile-first*, as *media queries* usam `min-width` e vão ao final da folha, porque não alteram especificidade;
- o ponto de quebra sai de onde o layout fica ruim, e não de uma lista de aparelhos;
- a 320px de largura não pode haver rolagem horizontal (WCAG 1.4.10);
- Bootstrap: 12 colunas e utilitárias, ao custo de 227 KB e de HTML cheio de classes de apresentação. Aqui, conteúdo de leitura.